

REFERENCE NOTES

Processes & OTP

spawn · send/receive · Task · Agent · GenServer
Supervisor · Application · DynamicSupervisor

Elixir Training Programme

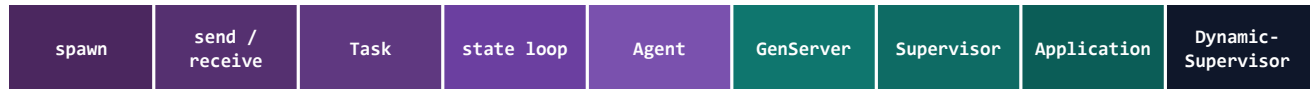
Elixir · processes, message passing and OTP

Reference notes · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

PROCESSES & OTP · REFERENCE NOTES

From a bare process to a supervised application

Everything that runs in Elixir runs inside a process. These notes climb one rung at a time: create a process, talk to it, get a result back, keep state in it, and finally let OTP start, watch and restart it for you. Each rung is built from the one below.



raw processes → one-off work → state → supervision

Contents

#	Topic	What it covers
1	spawn	Creating a process; spawn_link and spawn_monitor; links vs monitors
2	send and receive	Message passing, the mailbox, selective receive, timeouts
3	Task	Fire-and-forget, async/await, async_stream, Task.Supervisor
4	State	Holding state in a recursive receive loop
5	Agent	A process that only holds state: get, update, get_and_update
6	GenServer	Client API vs server callbacks; call, cast, info, terminate
7	Supervisor	Child specs, restart strategies, a restart you can watch
8	Application	mix.exs, the Application module, the supervision tree at boot
9	DynamicSupervisor	Starting and stopping children at runtime

HOW TO READ Every output block marked **REAL OUTPUT** was produced by running the code on Elixir 1.20.4 / Erlang OTP 29. PIDs and references (**#PID<0.97.0>**, **#Reference<...>**) are different on every run.

1

spawn

The most basic way to create a new process

IEX

```
pid = spawn(fn ->
  IO.puts("Hello from process #{inspect(self())}")
end)
```

- **Extremely lightweight** — thousands or millions can run.
- **Isolated memory** — no shared state.
- Returns a **PID** (process identifier).
- The process **dies when the function finishes** (or crashes).

LIB/APP/PROCESS.EX

```
defmodule App.Process do
  def start_process do
    pid =
      spawn(fn ->
        IO.puts("Hello from process #{inspect(self())}")
      end)

    IO.inspect(pid)
  end
end
```

REAL OUTPUT · ELIXIR 1.20.4 / OTP 29

```
#PID<0.97.0>
Hello from process #PID<0.97.0>
```

spawn_link/1 and spawn_monitor/1

There are also **spawn_link/1** and **spawn_monitor/1** (also available as **/3** variants). They are the two main ways to start a process **while establishing a relationship with the caller**.

spawn_link/1 (and /3)

- Atomically spawns a process and **links** it to the caller.
- Returns just the **pid**.
- The link is **bidirectional** (symmetric).
- When one process exits (for any reason other than **:normal**), it sends an exit signal to the other. Unless the receiver is trapping exits (**Process.flag(:trap_exit, true)**), the receiver also exits with the same reason.
- The classic “suicide pact” / failure-propagation mechanism.

spawn_monitor/1 (and /3)

- Atomically spawns a process and **monitors** it.
- Returns a tuple **{pid, monitor_ref}**.
- The monitor is **unidirectional** (asymmetric): only the caller watches.
- When the monitored process exits, the caller receives a regular message **{:DOWN, ref, :process, pid, reason}**.
- The caller does **not** exit. It just gets notified.

SPAWN_MONITOR IN IEX

```
{pid, ref} = spawn_monitor(fn -> raise "boom" end)

receive do
  {:DOWN, ^ref, :process, ^pid, reason} ->
    IO.inspect(reason) # the exit reason
end
```

Side by side

Aspect	spawn_link	spawn_monitor
Relationship	Bidirectional link	Unidirectional monitor
Failure behaviour	Propagates exit (can kill caller)	Sends :DOWN message only
Return value	pid	{pid, reference}
Trap exits needed?	Yes, if you want to survive	No
Typical use	Tightly coupled processes, supervisors	Observing / managing other processes without dying with them

When to use which

SPAWN_LINK Use **spawn_link** when the processes **truly depend on each other** — if one dies, the other should also die, or you want to trap the exit and react.

SPAWN_MONITOR Use **spawn_monitor** when you only need to be **informed** that another process has terminated, without risking your own process dying.

Demo: both relationships, one module

The notes list the expected output of a demo module; this is that module. The linked example traps exits first, so the caller survives the crash.

LIB/LINK_DEMO.EX

```
defmodule LinkDemo do
  def spawn_link_example do
    IO.puts("=== spawn_link example ===")
    Process.flag(:trap_exit, true)          # survive the linked crash

    pid =
      spawn_link(fn ->
        IO.puts("Linked process started (pid: #{inspect(self())})")
        raise "boom from linked process"
      end)

    IO.puts("Spawned linked process: #{inspect(pid)}")

    receive do
      {:EXIT, ^pid, reason} ->
        IO.puts("Received EXIT from #{inspect(pid)}")
        IO.puts("Reason: #{inspect(reason)}")
    end

    IO.puts("Current process still alive")
  end

  def spawn_monitor_example do
    IO.puts("\n=== spawn_monitor example ===")

    {pid, ref} =
      spawn_monitor(fn ->
        IO.puts("Monitored process started (pid: #{inspect(self())})")
        raise "boom from monitored process"
      end)

    IO.puts("Spawned monitored process: #{inspect(pid)}")
    IO.puts("Monitor reference: #{inspect(ref)}")

    receive do
      {:DOWN, ^ref, :process, ^pid, reason} ->
        IO.puts("Received DOWN from #{inspect(pid)}")
        IO.puts("Reason: #{inspect(reason)}")
    end

    IO.puts("Current process still alive")
  end
end
```

REAL OUTPUT · STACKTRACES SHORTENED TO [...]

```

=== spawn_link example ===
Linked process started (pid: #PID<0.101.0>)
Spawned linked process: #PID<0.101.0>
Received EXIT from #PID<0.101.0>
Reason: {%RuntimeError{message: "boom from linked process"}, [...]}
Current process still alive

=== spawn_monitor example ===
Spawned monitored process: #PID<0.102.0>
Monitored process started (pid: #PID<0.102.0>)
Monitor reference: #Reference<0.3160874833.952369156.162751>
Received DOWN from #PID<0.102.0>
Reason: {%RuntimeError{message: "boom from monitored process"}, [...]}
Current process still alive

```

Key points shown in the module

- **spawn_link** → you get an `{:EXIT, pid, reason}` message **only because we trapped exits**.
- **spawn_monitor** → you get a `{:DOWN, ref, :process, pid, reason}` message and the caller never dies.
- The reason for a crash is `{exception, stacktrace}`.
- Lines printed by two processes can interleave in a different order from run to run. The crashing process also writes an `[error] Process #PID<...> raised an exception` report to the logger.

2

send and receive

Processes communicate only by message passing

SENDER AND RECEIVER

```

# Sender
send(pid, {:hello, "world"})

# Receiver
receive do
  {:hello, msg} -> IO.puts("Got: #{msg}")
  other -> IO.puts("Unexpected: #{inspect(other)}")
after
  5_000 -> IO.puts("Timeout")
end

```

REAL OUTPUT · ELIXIR 1.20.4 / OTP 29

```
Got: world
```

- Messages are **copied** and go into the process **mailbox**.
- **receive** is **selective** and pattern-matches.
- **Non-blocking for the sender**; the receiver waits (or times out via **after**).

3

Task

Higher-level abstraction for one-off asynchronous work

Tasks build on top of the spawn functions to provide **better error reports and introspection**.

THE THREE SHAPES

```
# Fire and forget
Task.start(fn -> do_work() end)

# Await the result
task = Task.async(fn -> expensive_calculation() end)
result = Task.await(task, 10_000)

# Or with a supervisor
Task.Supervisor.start_child(MyApp.TaskSupervisor, fn -> ... end)
```

Common uses

- Parallel map (**Task.async_stream**)
- Background jobs
- Timeouts and cancellation support

Common patterns

- **Task.async/1** + **Task.await/2**
- **Task.async_stream/3** for concurrent mapping over a collection
- Supervised tasks via **Task.Supervisor**

A complete, ready-to-run module

TASK_EXAMPLES.EX

```
defmodule TaskExamples do
  @moduledoc """
  Full working examples of the three main Task APIs:

  1. Fire-and-forget → Task.start/1
  2. Await a result → Task.async/1 + Task.await/2
  3. Supervised task → Task.Supervisor.start_child/2
  """

  # -----
  # 1. Fire and forget – Task.start/1
  # -----
  def fire_and_forget do
    IO.puts("\n=== 1. Fire and forget (Task.start) ===")

    {:ok, pid} =
      Task.start(fn ->
        IO.puts(" [background] Starting work in #{inspect(self())}")
        :timer.sleep(800)
        IO.puts(" [background] Work finished!")
      end)

    IO.puts("Spawned fire-and-forget task: #{inspect(pid)}")
    IO.puts("Main process continues immediately...")
    :timer.sleep(1200) # just so we can see the background message
    IO.puts("Main process finished.\n")
  end
end
```

```

end

# -----
# 2. Await the result - Task.async + Task.await
# -----
def await_result do
  IO.puts("=== 2. Await the result (Task.async + Task.await) ===")

  task =
    Task.async(fn ->
      IO.puts(" [async] Starting expensive calculation in #{inspect(self())}")
      :timer.sleep(1000)
      result = 42 * 42
      IO.puts(" [async] Calculation done → #{result}")
      result
    end)

  IO.puts("Task started, now waiting for the result...")
  result = Task.await(task, 5_000) # timeout 5 seconds
  IO.puts("Got the result: #{result}\n")
end

# -----
# 3. Supervised task - Task.Supervisor.start_child
# -----
def supervised_task do
  IO.puts("=== 3. Supervised task (Task.Supervisor) ===")

  # Start a temporary Task.Supervisor for this demo
  {:ok, supervisor} = Task.Supervisor.start_link(name: :demo_task_sup)

  # Start a supervised task (linked to the supervisor)
  {:ok, pid} =
    Task.Supervisor.start_child(supervisor, fn ->
      IO.puts(" [supervised] Task started in #{inspect(self())}")
      :timer.sleep(700)
      IO.puts(" [supervised] Task completed successfully")
    end)

  IO.puts("Supervised task PID: #{inspect(pid)}")
  :timer.sleep(1000)

  # You can also start a task that returns a value and await it
  task =
    Task.Supervisor.async(supervisor, fn ->
      :timer.sleep(400)
      "hello from supervised async"
    end)

  result = Task.await(task)
  IO.puts("Supervised async result: #{result}")

  # Clean up the temporary supervisor
  Supervisor.stop(supervisor)
  IO.puts("Demo finished.\n")
end

```

```

end

# -----
# Run all examples
# -----
def run_all do
  fire_and_forget()
  await_result()
  supervised_task()
end
end

```

HOW TO RUN IT

```

# In iex
c("task_examples.ex") # or just paste the module
TaskExamples.run_all()

```

REAL OUTPUT · ELIXIR 1.20.4 / OTP 29

```

=== 1. Fire and forget (Task.start) ===
[background] Starting work in #PID<0.102.0>
Spawned fire-and-forget task: #PID<0.102.0>
Main process continues immediately...
[background] Work finished!
Main process finished.

=== 2. Await the result (Task.async + Task.await) ===
Task started, now waiting for the result...
[async] Starting expensive calculation in #PID<0.103.0>
[async] Calculation done → 1764
Got the result: 1764

=== 3. Supervised task (Task.Supervisor) ===
[supervised] Task started in #PID<0.105.0>
Supervised task PID: #PID<0.105.0>
[supervised] Task completed successfully
Supervised async result: hello from supervised async
Demo finished.

```

WATCH OUT The main process and the task print concurrently, so the first line of each pair can swap places between runs. The results (**1764**, the supervised string) never change.

Quick reference

API	Purpose	Returns	Linked?	Common use
<code>Task.start/1</code>	Fire-and-forget	<code>{:ok, pid}</code>	No	Side-effects only
<code>Task.async/1 + Task.await/2</code>	Get a result	<code>%Task{} → value</code>	Yes (to caller)	One-off calculations
<code>Task.Supervisor.start_child/2</code>	Supervised fire-and-forget	<code>{:ok, pid}</code>	Yes (to supervisor)	Background jobs under supervision
<code>Task.Supervisor.async/2</code>	Supervised + awaitable	<code>%Task{} → value</code>	Yes (to supervisor)	Awaitable work under supervision

4

State

The classic pattern, before OTP abstractions

A process holds state in a **recursive loop**: the state is the argument of the next call, and requests arrive through the mailbox.

THE RECEIVE LOOP

```
def loop(state) do
  receive do
    {:get, from} ->
      send(from, state)
      loop(state)
    {:put, new_state} ->
      loop(new_state)
  end
end
```

USING IT

```
pid = spawn(fn -> loop(0) end)
send(pid, {:put, 7})
send(pid, {:get, self()})
receive do v -> v end
# => 7
```

NEXT OTP provides better tools for this: **Agent**, **GenServer**, etc. — next.

5

Agent

Simple key-value / state holder — best for very simple shared state

- Internally an Agent **is a GenServer**.
- Prefer **GenServer** when you need more control (timeouts, casting, complex logic).

LIB/COUNTER.EX

```
defmodule Counter do
  @moduledoc """
  A simple stateful counter implemented with Agent.
  """

  # Start the Agent (holds the state)
  def start_link(initial_value \\ 0) do
    Agent.start_link(fn -> initial_value end, name: __MODULE__)
  end

  # Read the current state
  def value do
    Agent.get(__MODULE__, fn state -> state end)
  end

  # Update the state (increment)
  def increment(by \\ 1) do
    Agent.update(__MODULE__, fn state -> state + by end)
  end

  # Update the state (decrement)
  def decrement(by \\ 1) do
    Agent.update(__MODULE__, fn state -> state - by end)
  end

  # Reset the state
  def reset(to \\ 0) do
    Agent.update(__MODULE__, fn _state -> to end)
  end

  # Get and update in one step (returns the old value)
  def get_and_increment(by \\ 1) do
    Agent.get_and_update(__MODULE__, fn state ->
      {state, state + by} # {return_value, new_state}
    end)
  end
end
```

HOW TO USE IT IN IEX

```

{:ok, _pid} = Counter.start_link(10)  # 1. start with an initial value

Counter.value()                        # 2. read
# => 10

Counter.increment()                   # 3. increment
Counter.value()
# => 11
Counter.increment(5)
Counter.value()
# => 16

Counter.decrement(3)                  # 4. decrement
Counter.value()
# => 13

old = Counter.get_and_increment(2)     # 5. old value + update, atomically
# => 13
Counter.value()
# => 15

Counter.reset()                       # 6. reset
Counter.value()
# => 0

```

What's happening under the hood

Function	What it does	Agent API used
start_link	Creates a process that holds the state	Agent.start_link/2
value	Reads the current state (does not change it)	Agent.get/2
increment / decrement / reset	Changes the state	Agent.update/2
get_and_increment	Reads and updates atomically	Agent.get_and_update/2

Key points about Agent

- An Agent is **just a process that holds state**.
- State is completely **private** to that process (no shared memory).
- All communication happens by **sending messages** (get, update, get_and_update).
- Because it's a process, it is **isolated** — a crash in the Agent doesn't crash the caller (unless you link them).
- Agents are perfect for simple state. For callbacks, **handle_call**, **handle_cast** etc. use a **GenServer** instead.

6

GenServer

The workhorse of OTP — state, sync/async calls and lifecycle

LIB/COUNTER_SERVER.EX

```

defmodule CounterServer do
  use GenServer

  # =====
  # Client API (what the outside world calls)
  # =====

  def start_link(initial_value \\ 0) do
    GenServer.start_link(__MODULE__, initial_value, name: __MODULE__)
  end

  def value do
    GenServer.call(__MODULE__, :value)
  end

  def increment(by \\ 1) do
    GenServer.cast(__MODULE__, {:increment, by})
  end

  def decrement(by \\ 1) do
    GenServer.cast(__MODULE__, {:decrement, by})
  end

  def reset(to \\ 0) do
    GenServer.cast(__MODULE__, {:reset, to})
  end

  def get_and_increment(by \\ 1) do
    GenServer.call(__MODULE__, {:get_and_increment, by})
  end

  def stop do
    GenServer.stop(__MODULE__)
  end

  # =====
  # Server Callbacks (runs inside the GenServer process)
  # =====

  @impl true
  def init(initial_value) do
    # State is just an integer in this example
    {:ok, initial_value}
  end

  # Synchronous calls (caller waits for the reply)
  @impl true
  def handle_call(:value, _from, state) do
    {:reply, state, state}
  end

```

```
end

def handle_call({:get_and_increment, by}, _from, state) do
  new_state = state + by
  {:reply, state, new_state} # reply with old value, keep new state
end

# Asynchronous casts (fire-and-forget)
@impl true
def handle_cast({:increment, by}, state) do
  {:noreply, state + by}
end

def handle_cast({:decrement, by}, state) do
  {:noreply, state - by}
end

def handle_cast({:reset, to}, _state) do
  {:noreply, to}
end

# Optional: handle unexpected messages
@impl true
def handle_info(msg, state) do
  IO.puts("Unexpected message: #{inspect(msg)}")
  {:noreply, state}
end

# Optional: clean-up when the process terminates
@impl true
def terminate(reason, state) do
  IO.puts("CounterServer terminating. Reason: #{inspect(reason)}, last state: #{state}")
  :ok
end

end
```

HOW TO USE / RUN IT

```

{:ok, pid} = CounterServer.start_link(10)
# => {:ok, #PID<0.122.0>}

CounterServer.value()           # synchronous (call)
# => 10

CounterServer.increment()       # asynchronous (cast) - returns :ok at once
CounterServer.value()
# => 11
CounterServer.increment(5)
CounterServer.value()
# => 16
CounterServer.decrement(3)
CounterServer.value()
# => 13

old = CounterServer.get_and_increment(2)  # old value + update in one step
# => 13
CounterServer.value()
# => 15

CounterServer.reset(0)
CounterServer.value()
# => 0

CounterServer.stop()            # terminate/2 runs
# CounterServer terminating. Reason: :normal, last state: 0
# => :ok

```

The parts of a GenServer

Concept	How it appears in the code	Purpose
use GenServer	Top of the module	Brings in the behaviour
Client API	start_link, value, increment...	Nice public interface
init/1	Sets the initial state	Runs once when the process starts
handle_call/3	Synchronous requests (caller waits)	Used for value and get_and_increment
handle_cast/2	Asynchronous requests (fire-and-forget)	Used for increment, decrement, reset
handle_info/2	Catches unexpected messages	Good practice — send(CounterServer, :ping) prints "Unexpected message: :ping"
terminate/2	Clean-up callback	Called when the process stops

7

Supervisors

Monitor children and restart them according to a strategy

THE SHAPE

```
children = [
  {Counter, 0},
  {Task.Supervisor, name: MyApp.TaskSupervisor}
]

opts = [strategy: :one_for_one, name: MyApp.Supervisor]
Supervisor.start_link(children, opts)
```

Common strategies

Strategy	When one child crashes...
:one_for_one	restart only the crashed child
:one_for_all	restart all children
:rest_for_one	restart the crashed one and those started after it

A full example

1. THE GENSER (COUNTER)

```
defmodule Counter do
  use GenServer

  # ----- Client API -----
  def start_link(initial_value) do
    GenServer.start_link(__MODULE__, initial_value, name: __MODULE__)
  end

  def value, do: GenServer.call(__MODULE__, :value)
  def increment(by \\ 1), do: GenServer.cast(__MODULE__, {:increment, by})
  def decrement(by \\ 1), do: GenServer.cast(__MODULE__, {:decrement, by})
  def reset(to \\ 0), do: GenServer.cast(__MODULE__, {:reset, to})

  # ----- Server Callbacks -----
  @impl true
  def init(initial_value), do: {:ok, initial_value}

  @impl true
  def handle_call(:value, _from, state), do: {:reply, state, state}

  @impl true
  def handle_cast({:increment, by}, state), do: {:noreply, state + by}
  def handle_cast({:decrement, by}, state), do: {:noreply, state - by}
  def handle_cast({:reset, to}, _state), do: {:noreply, to}
end
```

2. THE APPLICATION SUPERVISOR

```
defmodule MyApp.Supervisor do
  use Supervisor

  def start_link(opts \\ []) do
    Supervisor.start_link(__MODULE__, opts, name: __MODULE__)
  end

  @impl true
  def init(_opts) do
    children = [
      # Start Counter with initial value 0
      {Counter, 0},

      # Start a Task.Supervisor
      {Task.Supervisor, name: MyApp.TaskSupervisor}
    ]

    # one_for_one = if one child dies, only that child is restarted
    opts = [strategy: :one_for_one]
    Supervisor.init(children, opts)
  end
end
```

3. APPLICATION ENTRY POINT (OPTIONAL BUT RECOMMENDED)

```
defmodule MyApp.Application do
  use Application

  @impl true
  def start(_type, _args) do
    MyApp.Supervisor.start_link()
  end
end
```

HOW TO RUN IT IN IEX

```
{:ok, _} = MyApp.Supervisor.start_link() # start the whole supervision tree

Counter.value()
# => 0
Counter.increment(5)
Counter.value()
# => 5
Counter.decrement(2)
Counter.value()
# => 3

# Use the Task.Supervisor
task = Task.Supervisor.async(MyApp.TaskSupervisor, fn ->
  :timer.sleep(500)
  42 * 2
end)
Task.await(task)
# => 84

# Fire-and-forget under the supervisor
Task.Supervisor.start_child(MyApp.TaskSupervisor, fn ->
  IO.puts("Background work running under supervision")
end)
```

Quick test of restart behaviour

IEX

```
Process.whereis(Counter) |> Process.exit(:kill) # kill the Counter process
:timer.sleep(50)                               # let the supervisor restart it

Counter.value()                                # alive again, initial state
# => 0
```

VERIFIED Measured: the Counter came back under a **new PID** (**#PID<0.130.0>** → **#PID<0.134.0>**) with its **initial state 0** — the 3 it held before the kill is gone. A restart gives you a fresh process, not the old one.

8

Applications

A reusable component with a supervision tree

MIX.EXS

```
def application do
  [
    extra_applications: [:logger],
    mod: {MyApp.Application, []}
  ]
end
```

A TYPICAL APPLICATION MODULE (PHOENIX-STYLE CHILDREN)

```
defmodule MyApp.Application do
  use Application

  def start(_type, _args) do
    children = [
      MyApp.Repo,
      {Phoenix.PubSub, name: MyApp.PubSub},
      MyAppWeb.Endpoint
    ]

    Supervisor.start_link(children, strategy: :one_for_one, name: MyApp.Supervisor)
  end
end
```

NOTE `Application.start/1` (or `mix run`) starts the **whole tree**.

A complete project

PROJECT LAYOUT

```
my_app/
├─ mix.exs
├─ lib/
│   └─ my_app/
│       ├── application.ex
│       ├── supervisor.ex
│       └─ counter.ex
└─ my_app.ex
```

1. MIX.EXS

```
defmodule MyApp.MixProject do
  use Mix.Project

  def project do
    [
      app: :my_app,
      version: "0.1.0",
      elixir: "~> 1.15",
      start_permanent: Mix.env() == :prod,
      deps: deps()
    ]
  end

  # This is the important part - tells OTP which module is the Application
  def application do
    [
      extra_applications: [:logger],
      mod: {MyApp.Application, []}
    ]
  end

  defp deps do
    []
  end
end
```

2. LIB/MY_APP/APPLICATION.EX

```
defmodule MyApp.Application do
  @moduledoc false
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      # Start the Counter GenServer with initial value 0
      {Counter, 0},

      # Start a Task.Supervisor
      {Task.Supervisor, name: MyApp.TaskSupervisor}
    ]

    # :one_for_one → if one child dies, only that one is restarted
    opts = [strategy: :one_for_one, name: MyApp.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

3. LIB/MY_APP/COUNTER.EX

```
defmodule Counter do
  use GenServer

  # ----- Client API -----
  def start_link(initial_value) do
    GenServer.start_link(__MODULE__, initial_value, name: __MODULE__)
  end

  def value, do: GenServer.call(__MODULE__, :value)
  def increment(by \\ 1), do: GenServer.cast(__MODULE__, {:increment, by})
  def decrement(by \\ 1), do: GenServer.cast(__MODULE__, {:decrement, by})
  def reset(to \\ 0), do: GenServer.cast(__MODULE__, {:reset, to})

  # ----- Server Callbacks -----
  @impl true
  def init(initial_value), do: {:ok, initial_value}

  @impl true
  def handle_call(:value, _from, state), do: {:reply, state, state}

  @impl true
  def handle_cast({:increment, by}, state), do: {:noreply, state + by}
  def handle_cast({:decrement, by}, state), do: {:noreply, state - by}
  def handle_cast({:reset, to}, _state), do: {:noreply, to}
end
```

4. LIB/MY_APP.EX (OPTIONAL HELPER)

```
defmodule MyApp do
  @moduledoc """
  Public API for the application.
  """

  def value, do: Counter.value()
  def increment(by \\ 1), do: Counter.increment(by)
  def decrement(by \\ 1), do: Counter.decrement(by)
  def reset(to \\ 0), do: Counter.reset(to)

  def run_background_job(fun) when is_function(fun, 0) do
    Task.Supervisor.start_child(MyApp.TaskSupervisor, fun)
  end
end
```

HOW TO USE IT - SHELL

```
# Create the project (if you haven't already)
mix new my_app --sup
# Then replace the generated files with the ones above

# Start the application
iex -S mix
```

INSIDE IEX

```
# The application is already started by OTP
Counter.value()
# => 0

Counter.increment(10)
Counter.value()
# => 10

# Use the Task.Supervisor
task = Task.Supervisor.async(MyApp.TaskSupervisor, fn ->
  :timer.sleep(500)
  :done
end)
Task.await(task)
# => :done

# Or via the helper
MyApp.run_background_job(fn -> IO.puts("Hello from supervised task") end)
```

What an Application does

Component	Responsibility
mix.exs → application/0	Declares the entry module (MyApp.Application)
MyApp.Application	Starts the top-level Supervisor when the app boots
Supervisor	Starts & monitors all children (Counter, Task.Supervisor, ...)
Children	Your GenServers, Agents, Task.Supervisors, etc.

AT BOOT When you run `iex -S mix` or start the release, OTP automatically calls `MyApp.Application.start/2`, which starts the whole supervision tree.

9

DynamicSupervisor

Children started at runtime — the modern replacement for `:simple_one_for_one`

1. COUNTER GENSERVER - STARTED BY PID, NOT BY NAME

```
defmodule Counter do
  use GenServer

  def start_link(initial_value) do
    GenServer.start_link(__MODULE__, initial_value)
  end

  def value(pid), do: GenServer.call(pid, :value)
  def increment(pid, by \\ 1), do: GenServer.cast(pid, {:increment, by})
  def decrement(pid, by \\ 1), do: GenServer.cast(pid, {:decrement, by})

  @impl true
  def init(initial_value), do: {:ok, initial_value}

  @impl true
  def handle_call(:value, _from, state), do: {:reply, state, state}

  @impl true
  def handle_cast({:increment, by}, state), do: {:noreply, state + by}
  def handle_cast({:decrement, by}, state), do: {:noreply, state - by}
end
```

2. THE DYNAMICSUPERVISOR

```
defmodule MyApp.DynamicSupervisor do
  use DynamicSupervisor

  def start_link(init_arg) do
    DynamicSupervisor.start_link(__MODULE__, init_arg, name: __MODULE__)
  end

  @impl true
  def init(_init_arg) do
    # :one_for_one is the only strategy DynamicSupervisor supports
    DynamicSupervisor.init(strategy: :one_for_one)
  end

  # ----- Helper to start a Counter under the DynamicSupervisor -----
  def start_counter(initial_value \\ 0) do
    child_spec = %{
      id: Counter,
      start: {Counter, :start_link, [initial_value]},
      restart: :temporary # or :permanent / :transient
    }

    DynamicSupervisor.start_child(__MODULE__, child_spec)
  end

  # Convenience: start many counters
  def start_many(count, initial_value \\ 0) do
    Enum.map(1..count, fn _ -> start_counter(initial_value) end)
  end
end
```

3. THE APPLICATION THAT STARTS IT

```
defmodule MyApp.Application do
  use Application

  @impl true
  def start(_type, _args) do
    children = [
      # Start the DynamicSupervisor (it starts with ZERO children)
      {MyApp.DynamicSupervisor, []}
    ]

    opts = [strategy: :one_for_one, name: MyApp.Supervisor]
    Supervisor.start_link(children, opts)
  end
end
```

HOW TO USE IT - IEX -S MIX

```
# Start individual counters dynamically
{:ok, pid1} = MyApp.DynamicSupervisor.start_counter(10)
{:ok, pid2} = MyApp.DynamicSupervisor.start_counter(100)

Counter.value(pid1)    # => 10
Counter.increment(pid1, 5)
Counter.value(pid1)    # => 15
Counter.value(pid2)    # => 100

# Start many at once
results = MyApp.DynamicSupervisor.start_many(3, 0)
# => [ok: #PID<0.144.0>, ok: #PID<0.145.0>, ok: #PID<0.146.0>]

# See all running children
DynamicSupervisor.which_children(MyApp.DynamicSupervisor)
# => [{:undefined, #PID<0.142.0>, :worker, [Counter]}, ...]

# Count how many children are alive
DynamicSupervisor.count_children(MyApp.DynamicSupervisor)
# => %{active: 5, workers: 5, supervisors: 0, specs: 5}

# Stop a specific child
DynamicSupervisor.terminate_child(MyApp.DynamicSupervisor, pid1)
# => :ok      (count_children now reports active: 4)
```

Static Supervisor vs DynamicSupervisor

Feature	Supervisor (static)	DynamicSupervisor
Children defined at	Compile / start time	Runtime
Starting new children	Normally fixed at start (start_child/2 exists, rarely used)	start_child/2 anytime
Strategy	:one_for_one, :one_for_all, :rest_for_one	Only :one_for_one
Typical use	Fixed set of processes (Counter, Registry, etc.)	Workers that come and go (connections, jobs, sessions...)
Child restart	Configurable per child	Configurable per child

VERIFIED A DynamicSupervisor **starts with zero children** — `count_children` reports **active: 0** until the first `start_child/2`.